```
    })
    cy.get("@dayTwoConcerts").each((item) => {
      cy.wrap(item).should("not.be.visible")
    })
    cy.get("@dayThreeConcerts").each((item) => {
      cy.wrap(item).should("not.be.visible")
    })
  })

  it("shows all on show all", () => {
    cy.get("@dayOne").click()
    cy.get("@dayTwo").click()
    cy.contains("Show All").click()
    cy.get("@dayOne").should("not.have.class", "border-red-700")
    cy.get("@dayTwo").should("not.have.class", "border-red-700")
    cy.get("@dayOneConcerts").each((item) => {
      cy.wrap(item).should("be.visible")
    })
    cy.get("@dayTwoConcerts").each((item) => {
      cy.wrap(item).should("be.visible")
    })
    cy.get("@dayThreeConcerts").each((item) => {
      cy.wrap(item).should("be.visible")
    })
  })
})
```

Because we changed the concert listing to have the day headers managed in CSS, we don't have easy `div` elements for each day, so we need to get a little fancier.

The test section here starts with a `beforeEach`, which starts by using `visit` to get to the schedule page. We then need to do a little selector manipulation to get the DOM IDs for each of the three days that are in the seeded data—the seeded data hard-codes the dates so that we can do this very thing. Similarly, we use the same data attributes we use in the dynamic CSS to build up a list of the actual concerts associated with each day. (Because we don't use `first` in these lines, Cypress is still going to treat the results as a list of elements.)

The first test kind of gives the pattern of how we can use Cypress to check the visibility status. We use `cy.get` to grab each alias and then call `.each` to iterate over